OOP in Java

Slide 4 – Loops in Java: Doing Things Again and Again

Ishan Shrestha

Why Use Loops in Programming?

- **What Is a Loop?**
- Analogy 1: Imagine you have a basket of laundry with 5 socks to fold. Instead of saying, "Fold this sock" 5 separate times, you can just say, "Fold a sock" and repeat that until all the socks are folded.
- **Analogy 2**: Imagine you're waiting for an elevator. Every time it arrives, you check if it's going to your floor. If it's not, you let it go and wait for the next one, repeating the process until you get the right elevator.
- In programming, a loop is like that: it allows you to automatically repeat the same task—like folding socks or checking the elevator—until a condition is met (like all the socks are folded or the elevator arrives at your floor), without having to instruct the computer step by step each time.

Introducing the `for` Loop

- The for loop allows you to repeat a block of code a specific number of times. It's great when you know exactly how many times you want to repeat something.
- Basic Syntax
  - **Initialization**: Sets up a starting point, usually a variable like i = 0.
  - **Condition**: The loop runs as long as this is **true**.
  - **Update**: Changes the value after each loop, usually increases or decreases a variable.

```
for (initialization; condition; update) {
    // Code to run repeatedly
}
```

Example of a `for` Loop

- Explanation
  - **Initialization**: int i = 1; sets up the counter starting at 1.
  - **Condition**: i <= 5 ensures the loop runs as long as i is 5 or less.
  - **Update**: i++ increases i by 1 after every loop

```java
// Example: Print numbers from 1 to 5.
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

```
1
2
3
4
5
```

Understanding the Loop Flow

- **Step-by-Step**:
  - **Step 1**: i starts at 1.
  - **Step 2**: The computer checks if i <= 5. If true, it runs the code.
  - **Step 3**: After running the code, i increases by 1 (now i = 2), and the loop repeats.
  - **Step 4**: This continues until i becomes 6, and the condition i <= 5 becomes false. The loop stops.

The `while` Loop

- The while loop is another type of loop. It repeats a block of code **as long as a condition is true**. It’s useful when you don’t know in advance how many times the loop will run.

```java
// Basic Syntax
while (condition) {
    // Code to run while the condition is true
}
```

```java
// Example: Print numbers from 1 to 5 using a while loop.
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;  // Increase the value of i each time
}
```

When to Use `for` vs. `while`?

Use for: When you know exactly how many times the loop should run (e.g., counting from 1 to 5).

Use while: When you want the loop to keep going until something changes (e.g., keep asking a question until you get the correct answer).

Exercise

- **Task 1**: Use a for loop to print numbers from 1 to 10.
- **Task 2**: Use a while loop to print numbers from 1 to 10.

Breaking Out of a Loop

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        break;  // Exit the loop
    }
    System.out.println(i);
}

The loop will print 1 and 2, but when i becomes 3,
the break statement will exit the loop
```

- You can stop a loop early using the **break** statement.
- **Example**: Stop the loop when i reaches 3.

Skipping Iterations

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;  // Skip this iteration
    }
    System.out.println(i);
}

Explanation: The loop will print 1, 2, 4,
and 5. It will skip printing the number 3.
```

- You can skip the current iteration of a loop using the **continue** statement.
- **Example**: Skip printing the number 3.

Summary

- **Loops**: Used to repeat actions without writing the same code multiple times.
- `for` **Loop**: Repeats code a specific number of times.
- `while` **Loop**: Repeats code as long as a condition is true.
- `break` **and** `continue`: Use these to control the flow of your loops.